Software Requirements Specification

Sportify

Sports Database Management System

Document Version	1.0
Status	Draft
Date	February 28, 2026
Course	Database Management Systems (DBMS)

Team Members

Name	Role
A S M Raiyan	Frontend (ReactJS) & Backend System Overview
Ratul Hasan Anik	Backend API Routing (Python / Flask)
Mashrafi Jaman Jahin	External API Integration & Live Data Fetching
Arian Parvez Teshan	Authentication, RBAC & Security

All rights reserved. For academic use only.

0 Contents

Revision History **3**

1 Introduction **4**
1.1 Purpose . . . 4
1.2 Project Scope . . . 4
1.3 Definitions, Acronyms, and Abbreviations . . . 5
1.4 References . . . 5
1.5 Overview of Document . . . 5

2 Team Responsibilities **6**

3 System Architecture & Technologies **8**
3.1 High-Level Architecture . . . 8
3.2 Technology Stack . . . 8
3.3 Component Interaction Flow . . . 8

4 Functional Requirements **10**
4.1 User Registration & Authentication . . . 10
4.2 User Roles & Access Control . . . 10
4.3 Live Data Fetching & Synchronisation . . . 11
4.4 Sports Data — Matches & Live Scores . . . 11
4.5 Player Management . . . 11
4.6 Team & League Management . . . 12
4.7 Follow System . . . 12
4.8 Reviews & Comments . . . 13
4.9 Highlight Videos & Fan Events . . . 13

5 Non-Functional Requirements **14**
5.1 Performance . . . 14
5.2 Reliability & Availability . . . 14
5.3 Security . . . 14
5.4 Maintainability . . . 14
5.5 Scalability . . . 14
5.6 Usability . . . 15

6 Database Requirements & Schema Design **16**
6.1 Normalisation Goals . . . 16
6.2 Entity Descriptions . . . 16
6.2.1 Core Entities . . . 16
6.2.2 Engagement Entities . . . 16
6.2.3 Junction / Weak Entities . . . 16
6.3 Relational Schema (SQL DDL) . . . 16
6.4 Key Design Decisions . . . 24

7 API Design **25**

8 Authentication & Security Design **27**
8.1 Authentication Flow 27
8.2 RBAC Enforcement 27
8.3 Input Validation & Injection Prevention 27
8.4 Rate Limiting 27

9 External API Integration **28**
9.1 Candidate APIs 28
9.2 Data Sync Strategy 28
9.3 Upsert Pattern 28

10 Constraints & Assumptions **29**
10.1 Constraints 29
10.2 Assumptions 29

11 Use Cases (Summary) **30**

12 Testing Strategy **31**

13 Project Timeline (Indicative) **32**

0 Revision History

Version	Date	Description	Author
1.0	February 28, 2026	Initial draft of SRS document	All Members

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document provides a comprehensive, formal description of **Sportify**—a web-based sports management and live-update system. The document is intended for:

- **Developers** — to understand system behaviour and implementation constraints.
- **Database Designers** — to design a normalised relational schema.
- **Instructors / Evaluators** — to assess alignment with DBMS course objectives.
- **Testers** — to derive test cases from stated requirements.

1.2 Project Scope

Sportify is a full-stack web application that:

- Aggregates real-time sports data (football, cricket, wrestling) from third-party REST APIs.
- Stores, normalises, and indexes data in a **MySQL** relational database.
- Presents live scores, league standings, player statistics, and match schedules to end-users.
- Supports community features: reviews, comments, follow system, highlight videos, and fan-event registrations.
- Enforces role-based access for three user classes: **Admin**, **Team Manager**, and **Sports Fan**.

Out of Scope (v1.0):

- Mobile native applications (iOS / Android).
- Paid subscription / payment gateway integration.
- AI/ML-based prediction features.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification
DBMS	Database Management System
API	Application Programming Interface
REST	Representational State Transfer
RBAC	Role-Based Access Control
JWT	JSON Web Token
ORM	Object-Relational Mapping
FK	Foreign Key
PK	Primary Key
1NF/2NF/3NF	First / Second / Third Normal Form
BCNF	Boyce-Codd Normal Form
UI	User Interface

1.4 References

- IEEE Std 830-1998 — IEEE Recommended Practice for SRS.
- API-Sports Documentation: `https://api-sports.io`
- Flask Framework Docs: `https://flask.palletsprojects.com`
- ReactJS Docs: `https://react.dev`
- MySQL 8.x Reference Manual: `https://dev.mysql.com/doc/`

1.5 Overview of Document

Section 2 covers team responsibilities. Section 3 describes system architecture. Sections 4–5 enumerate functional and non-functional requirements. Section 6 provides complete database design with ER descriptions and schema. Section 8 covers authentication and security. Section 7 covers API design. Section 10 lists constraints and assumptions.

2 Team Responsibilities

All four members jointly own the database schema design, normalisation, and testing. Individual development responsibilities are divided as follows.

Member	Primary Role	Detailed Responsibilities
A S M Raiyan	Frontend & Backend Overview	• Implement ReactJS components, pages, and routing. • Design UI/UX wireframes and state management (Context API / Redux). • Coordinate backend integration from a system-level view. • Write end-to-end integration tests.
Ratul Hasan Anik	Backend API Routing	• Implement Flask Blueprints for all internal REST endpoints. • Define request validation, error handling, and response serialisation. • Write SQL queries / SQLAlchemy ORM models. • Manage database migration scripts.
Mashrafi Jaman Jahin	External API Integration	• Identify and integrate external sports data APIs. • Implement scheduled data-sync jobs (APScheduler / cron). • Handle rate-limiting, retry logic, and data transformation. • Map external JSON payloads to normalised DB tables.

Member	Primary Role	Detailed Responsibilities
Arian Parvez Teshan	Authentication & Security	• Implement JWT-based user authentication and session management. • Design and enforce RBAC (Admin, Team Manager, Sports Fan). • Apply OWASP security best practices (input sanitisation, HTTPS, CORS). • Manage password hashing (bcrypt) and token refresh flows.

3 System Architecture & Technologies

3.1 High-Level Architecture

Sportify follows a three-tier architecture:

1. **Presentation Tier** — ReactJS Single-Page Application (SPA) served via a web server (e.g. Nginx / Vercel).
2. **Application Tier** — Python Flask REST API, packaged as a WSGI application behind Gunicorn.
3. **Data Tier** — MySQL 8.x relational database with normalised schema.

A background **Scheduler Service** (APScheduler, integrated with Flask) periodically calls external sports APIs and upserts data into the database, decoupled from user-facing requests.

3.2 Technology Stack

Layer	Technology	Notes
Frontend	ReactJS 18	Vite build tool, React Router, Axios
State Management	Context API / Redux	For global auth and live-score state
Backend Framework	Python 3.11 / Flask 3	Blueprint-based modular structure
ORM	SQLAlchemy 2	Declarative models, Alembic migrations
Database	MySQL 8.x	InnoDB engine, UTF-8mb4 charset
Authentication	JWT (PyJWT)	Access + Refresh token pair
Password Hashing	bcrypt	Cost factor 12
External APIs	API-Sports / Cricbuzz	REST JSON, polled every 60 seconds during live events
Scheduler	APScheduler	Integrated with Flask app context
Version Control	Git + GitHub	Feature-branch workflow, PR reviews
Deployment (dev)	Docker Compose	Containers: flask-app, mysql, nginx

3.3 Component Interaction Flow

1. User opens the React SPA in a browser.
2. SPA sends an HTTPS request to the Flask API (e.g. `GET /api/v1/matches/live`).
3. Flask validates the JWT token, queries MySQL, and returns a JSON response.

4. In parallel, the Scheduler polls external APIs, transforms data, and upserts records into MySQL.
5. Real-time score updates are pushed to the SPA via **Server-Sent Events (SSE)** or WebSocket (optional enhancement).

4 Functional Requirements

Requirements are tagged **[FR-XXX]** and assigned a priority: **High**, **Medium**, or **Low**.

4.1 User Registration & Authentication

ID	Requirement	Priority
FR-01	The system shall allow new users to register with a unique email address, a display name, and a password.	High
FR-02	The system shall hash passwords using bcrypt before storing them in the database.	High
FR-03	The system shall issue a signed JWT access token (15-minute expiry) and a refresh token (7-day expiry) upon successful login.	High
FR-04	The system shall allow users to refresh their access token using a valid refresh token without re-entering credentials.	High
FR-05	The system shall allow users to log out, invalidating the current refresh token.	High
FR-06	The system shall support password reset via a time-limited emailed token.	Medium
FR-07	The system shall allow Admin users to manually assign or change user roles (Fan, Team Manager).	High

4.2 User Roles & Access Control

ID	Requirement	Priority
FR-08	Three roles shall exist: **Admin**, **Team Manager**, and **Sports Fan**. Every registered user is assigned exactly one role.	High
FR-09	**Admin** shall be able to: view all users, suspend/delete accounts, verify manually entered data, manage sports/leagues, and assign Team Manager roles.	High
FR-10	**Team Manager** shall be able to: update their team's squad details (player injury status, lineup), and view team performance analytics.	High
FR-11	**Sports Fan** shall be able to: view all public data, follow teams/players, post reviews and comments, register for fan events.	High

ID	Requirement	Priority
FR-12	Unauthenticated visitors shall be able to browse live scores, standings, and player stats in read-only mode.	Medium

4.3 Live Data Fetching & Synchronisation

ID	Requirement	Priority
FR-13	The system shall integrate at least one external sports REST API to fetch match data for football, cricket, and wrestling.	High
FR-14	The scheduler shall poll the external API every **60 seconds** for live matches and every **6 hours** for non-live data (fixtures, standings).	High
FR-15	The system shall upsert (INSERT ON DUPLICATE KEY UPDATE) fetched data, ensuring no duplicate records.	High
FR-16	The system shall log each sync job (timestamp, records fetched, errors) in a `sync_log` table.	Medium
FR-17	If the external API is unavailable, the system shall serve cached data from the database and display a staleness warning to the user.	Medium

4.4 Sports Data — Matches & Live Scores

ID	Requirement	Priority
FR-18	The system shall display a real-time scoreboard for live matches including: teams, current score, elapsed time, and match status (scheduled, live, finished).	High
FR-19	The system shall allow users to filter matches by sport, league, date, and status.	High
FR-20	The system shall display detailed match statistics (possession, wickets, fall of wickets, etc.) for each sport.	High
FR-21	The system shall store a complete match history searchable by team, player, and date range.	Medium

4.5 Player Management

ID	Requirement	Priority
FR-22	The system shall store player profiles: name, nationality, date of birth, position/role, current team, and profile image URL.	High
FR-23	The system shall display per-player statistics aggregated per season: goals, assists, batting average, wickets, etc.	High
FR-24	The system shall store and display player injury status (fit, injured, doubtful) with a last-updated timestamp.	High
FR-25	**Team Managers** shall be able to update injury status for players in their own team only.	High
FR-26	The system shall support community fan-submitted player ratings (1–5 stars) per match, averaged and displayed on the player profile.	Medium

4.6 Team & League Management

ID	Requirement	Priority
FR-27	The system shall store team profiles: name, country, founding year, stadium/home ground, logo URL, and associated league.	High
FR-28	The system shall display league standings: rank, team, played, won, drawn, lost, points, goal difference.	High
FR-29	The system shall support multiple sports under one system, each with their own league/tournament hierarchy.	High
FR-30	**Admin** shall be able to manually add or edit league and team records.	Medium

4.7 Follow System

ID	Requirement	Priority
FR-31	Authenticated users (all roles) shall be able to follow or unfollow any team.	High
FR-32	Authenticated users shall be able to follow or unfollow any player.	High
FR-33	The system shall display a personalised feed of followed teams' and players' latest events on the user's dashboard.	Medium

ID	Requirement	Priority
FR-34	The system shall notify followed-team fans (via on-site notification) when a live match begins.	Low

4.8 Reviews & Comments

ID	Requirement	Priority
FR-35	Authenticated Sports Fans shall be able to post a review (text + star rating 1–5) for any completed match.	High
FR-36	Authenticated users shall be able to post, edit (within 15 minutes), and delete their own comments on match and player pages.	High
FR-37	The system shall support threaded replies (one level deep) on comments.	Medium
FR-38	**Admin** shall be able to hide or delete any review or comment that violates community guidelines.	High
FR-39	The system shall display the average star rating aggregated from all reviews for a given match.	Medium

4.9 Highlight Videos & Fan Events

ID	Requirement	Priority
FR-40	The system shall store and display post-match highlight video links (YouTube embed or direct URL) associated with a match record.	Medium
FR-41	**Admin** shall be able to add, edit, and remove highlight video entries.	Medium
FR-42	The system shall allow Admin to create fan events (name, description, date, location, capacity).	Medium
FR-43	Authenticated Sports Fans shall be able to register for an event (subject to capacity).	Medium
FR-44	The system shall prevent duplicate registrations from the same user for the same event.	Medium

5 Non-Functional Requirements

5.1 Performance

- The API shall respond to 95% of read requests within **500 ms** under a load of 100 concurrent users.
- Live-score polling shall complete a full sync cycle within **10 seconds** per run.
- Database queries on indexed columns (`match_id`, `team_id`, `player_id`) shall complete within **50 ms**.

5.2 Reliability & Availability

- The system shall target **99%** uptime during live match windows.
- All database transactions shall be ACID-compliant using InnoDB engine.
- The scheduler shall implement exponential back-off on external API failure (max 5 retries).

5.3 Security

- All HTTP traffic shall be served over HTTPS (TLS 1.2+).
- Passwords shall never be stored in plaintext; bcrypt (cost 12) is mandatory.
- JWT tokens shall be signed with HS256 and stored in `httpOnly` cookies on the frontend.
- The system shall validate and sanitise all user inputs to prevent SQL Injection and XSS.
- CORS shall be restricted to the frontend origin domain only.
- Rate limiting shall be applied: 60 requests / minute per IP on public endpoints.

5.4 Maintainability

- Code shall follow PEP-8 (backend) and ESLint Airbnb rules (frontend).
- Database migrations shall be managed with Alembic; no raw schema changes without a migration file.
- All API routes shall be documented in OpenAPI 3.0 (Swagger).

5.5 Scalability

- The database schema shall support adding new sports without structural changes (sport is a configurable entity, not hardcoded).
- The Flask application shall be stateless to allow horizontal scaling behind a load balancer.

5.6 Usability

- The frontend shall be responsive and functional on screen widths from 360 px (mobile) to 1920 px (desktop).
- Live score pages shall auto-refresh without a full page reload.

6 Database Requirements & Schema Design

6.1 Normalisation Goals

The schema shall satisfy **Third Normal Form (3NF)** at minimum. Many-to-Many relationships (follow system, event registrations, player–match stats) shall be resolved using explicit junction tables with composite primary keys.

6.2 Entity Descriptions

6.2.1 Core Entities

- **User** — platform accounts with role.
- **Sport** — configurable top-level sport (football, cricket, wrestling . . .).
- **League / Tournament** — belongs to a Sport; has a season.
- **Team** — competes in one or more Leagues; has a home ground.
- **Player** — belongs to a Team; has sport-specific attributes.
- **Match** — between two Teams in a League; has status and score.

6.2.2 Engagement Entities

- **Review** — a User's star rating + text for a Match.
- **Comment** — text comment by a User on a Match or Player page; supports parent (thread).
- **Highlight** — video URL linked to a Match.
- **FanEvent** — Admin-created event with capacity.
- **EventRegistration** — junction between User and FanEvent.

6.2.3 Junction / Weak Entities

- **UserFollowTeam** — M:N between User and Team.
- **UserFollowPlayer** — M:N between User and Player.
- **PlayerMatchStat** — stores per-player, per-match statistics.
- **TeamLeague** — M:N between Team and League (a team can play in multiple leagues / seasons).
- **SyncLog** — audit table for scheduler runs.

6.3 Relational Schema (SQL DDL)

```
-- ============================================================
--  Sportify Database Schema
```

```sql
--  Engine: InnoDB | Charset: utf8mb4 | Collation: utf8mb4_unicode_ci
-- ==============================================================

CREATE DATABASE IF NOT EXISTS sportify
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

USE sportify;

-- --------------------------------------------------------------
-- 1. SPORT
-- --------------------------------------------------------------
CREATE TABLE Sport (
    sport_id    INT           UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    name        VARCHAR(100)  NOT NULL UNIQUE,
    description TEXT,
    created_at  TIMESTAMP     DEFAULT CURRENT_TIMESTAMP
);

-- --------------------------------------------------------------
-- 2. LEAGUE
-- --------------------------------------------------------------
CREATE TABLE League (
    league_id         INT          UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    sport_id          INT          UNSIGNED NOT NULL,
    name              VARCHAR(150) NOT NULL,
    country           VARCHAR(100),
    season            VARCHAR(20)  NOT NULL,    -- e.g. "2024-25"
    external_api_id   VARCHAR(100),             -- ID from external API
    created_at        TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_league_sport FOREIGN KEY (sport_id) REFERENCES Sport(sport_id)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    UNIQUE KEY uq_league_season (name, season)
);

-- --------------------------------------------------------------
-- 3. TEAM
-- --------------------------------------------------------------
CREATE TABLE Team (
    team_id           INT          UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    sport_id          INT          UNSIGNED NOT NULL,
    name              VARCHAR(150) NOT NULL,
    country           VARCHAR(100),
    founded_year      YEAR,
```

```sql
    home_ground        VARCHAR(200),
    logo_url           VARCHAR(500),
    external_api_id    VARCHAR(100),
    created_at         TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_team_sport FOREIGN KEY (sport_id) REFERENCES Sport(sport_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
);

-- ------------------------------------------------------------
-- 4. TEAM_LEAGUE  (junction: Team <-> League)
-- ------------------------------------------------------------
CREATE TABLE TeamLeague (
    team_id     INT UNSIGNED NOT NULL,
    league_id   INT UNSIGNED NOT NULL,
    PRIMARY KEY (team_id, league_id),
    CONSTRAINT fk_tl_team   FOREIGN KEY (team_id)   REFERENCES Team(team_id)   ON DELETE CASCADE,
    CONSTRAINT fk_tl_league FOREIGN KEY (league_id) REFERENCES League(league_id) ON DELETE CASCADE
);

-- ------------------------------------------------------------
-- 5. PLAYER
-- ------------------------------------------------------------
CREATE TABLE Player (
    player_id          INT           UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    team_id            INT           UNSIGNED,              -- NULL if free agent
    sport_id           INT           UNSIGNED NOT NULL,
    name               VARCHAR(150) NOT NULL,
    nationality        VARCHAR(100),
    date_of_birth      DATE,
    position_role      VARCHAR(100),                        -- e.g. "Forward", "Batsman"
    jersey_number      TINYINT       UNSIGNED,
    profile_image_url  VARCHAR(500),
    injury_status      ENUM('fit','injured','doubtful') DEFAULT 'fit',
    injury_updated_at  TIMESTAMP     NULL,
    external_api_id    VARCHAR(100),
    created_at         TIMESTAMP     DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_player_team  FOREIGN KEY (team_id)  REFERENCES Team(team_id)  ON DELETE SET NULL,
    CONSTRAINT fk_player_sport FOREIGN KEY (sport_id) REFERENCES Sport(sport_id) ON DELETE RESTRICT
);
```

```sql
-- ------------------------------------------------------------
-- 6. USER
-- ------------------------------------------------------------
CREATE TABLE User (
    user_id        INT           UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    email          VARCHAR(255) NOT NULL UNIQUE,
    display_name   VARCHAR(100) NOT NULL,
    password_hash  VARCHAR(255) NOT NULL,
    role           ENUM('admin','team_manager','fan') NOT NULL DEFAULT 'fan',
    team_managed   INT           UNSIGNED NULL,          -- FK to Team (for team_manager role)
    is_active      BOOLEAN       NOT NULL DEFAULT TRUE,
    created_at     TIMESTAMP     DEFAULT CURRENT_TIMESTAMP,
    last_login     TIMESTAMP     NULL,
    CONSTRAINT fk_user_team FOREIGN KEY (team_managed) REFERENCES Team(team_id)
        ON DELETE SET NULL ON UPDATE CASCADE
);

-- ------------------------------------------------------------
-- 7. REFRESH_TOKEN
-- ------------------------------------------------------------
CREATE TABLE RefreshToken (
    token_id     INT           UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    user_id      INT           UNSIGNED NOT NULL,
    token_hash   VARCHAR(512) NOT NULL UNIQUE,
    expires_at   TIMESTAMP     NOT NULL,
    revoked      BOOLEAN       DEFAULT FALSE,
    created_at   TIMESTAMP     DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_rt_user FOREIGN KEY (user_id) REFERENCES User(user_id) ON DELETE CASCADE
);

-- ------------------------------------------------------------
-- 8. MATCH
-- ------------------------------------------------------------
CREATE TABLE `Match` (
    match_id         INT           UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    league_id        INT           UNSIGNED NOT NULL,
    home_team_id     INT           UNSIGNED NOT NULL,
    away_team_id     INT           UNSIGNED NOT NULL,
    match_datetime   DATETIME      NOT NULL,
    venue            VARCHAR(200),
    status           ENUM('scheduled','live','finished','postponed','cancelled')
                     NOT NULL DEFAULT 'scheduled',
```

```sql
    home_score      SMALLINT     UNSIGNED DEFAULT 0,
    away_score      SMALLINT     UNSIGNED DEFAULT 0,
    elapsed_time    SMALLINT     UNSIGNED,        -- minutes elapsed (football), overs (cricket)
    extra_data      JSON,                          -- sport-specific raw data
    external_api_id VARCHAR(100),
    created_at      TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
    updated_at      TIMESTAMP    DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT fk_match_league    FOREIGN KEY (league_id) REFERENCES League(league_id),
    CONSTRAINT fk_match_home_team FOREIGN KEY (home_team_id) REFERENCES Team(team_id),
    CONSTRAINT fk_match_away_team FOREIGN KEY (away_team_id) REFERENCES Team(team_id),
    CONSTRAINT chk_teams CHECK (home_team_id <> away_team_id)
);

-- ------------------------------------------------------------
-- 9. PLAYER_MATCH_STAT  (junction: Player <-> Match)
-- ------------------------------------------------------------
CREATE TABLE PlayerMatchStat (
    stat_id       INT          UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    player_id     INT          UNSIGNED NOT NULL,
    match_id      INT          UNSIGNED NOT NULL,
    -- Generic stats (extend via extra_stats JSON for sport specifics)
    minutes_played SMALLINT    UNSIGNED DEFAULT 0,
    goals         TINYINT      UNSIGNED DEFAULT 0,
    assists       TINYINT      UNSIGNED DEFAULT 0,
    yellow_cards  TINYINT      UNSIGNED DEFAULT 0,
    red_cards     TINYINT      UNSIGNED DEFAULT 0,
    runs_scored   SMALLINT     UNSIGNED DEFAULT 0,   -- cricket
    wickets       TINYINT      UNSIGNED DEFAULT 0,   -- cricket
    extra_stats   JSON,
    UNIQUE KEY uq_player_match (player_id, match_id),
    CONSTRAINT fk_pms_player FOREIGN KEY (player_id) REFERENCES Player(player_id) ON DELETE CASCADE,
    CONSTRAINT fk_pms_match  FOREIGN KEY (match_id)  REFERENCES `Match`(match_id) ON DELETE CASCADE
);

-- ------------------------------------------------------------
-- 10. REVIEW
-- ------------------------------------------------------------
CREATE TABLE Review (
    review_id   INT          UNSIGNED AUTO_INCREMENT PRIMARY KEY,
```

```sql
    match_id     INT           UNSIGNED NOT NULL,
    user_id      INT           UNSIGNED NOT NULL,
    rating       TINYINT       UNSIGNED NOT NULL,
    body         TEXT,
    is_hidden    BOOLEAN       DEFAULT FALSE,
    created_at   TIMESTAMP     DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY uq_review_user_match (user_id, match_id),
    CONSTRAINT chk_rating CHECK (rating BETWEEN 1 AND 5),
    CONSTRAINT fk_review_match FOREIGN KEY (match_id) REFERENCES `Match`(match_id) ON DELETE CASCADE,
    CONSTRAINT fk_review_user  FOREIGN KEY (user_id)  REFERENCES User(user_id)  ON DELETE CASCADE
);

-- ------------------------------------------------------------
-- 11. COMMENT
-- ------------------------------------------------------------
CREATE TABLE Comment (
    comment_id   INT           UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    user_id      INT           UNSIGNED NOT NULL,
    entity_type  ENUM('match','player') NOT NULL,
    entity_id    INT           UNSIGNED NOT NULL,
    parent_id    INT           UNSIGNED NULL,          -- NULL = top-level comment
    body         TEXT          NOT NULL,
    is_hidden    BOOLEAN       DEFAULT FALSE,
    created_at   TIMESTAMP     DEFAULT CURRENT_TIMESTAMP,
    edited_at    TIMESTAMP     NULL,
    CONSTRAINT fk_comment_user   FOREIGN KEY (user_id) REFERENCES User(user_id)    ON DELETE CASCADE,
    CONSTRAINT fk_comment_parent FOREIGN KEY (parent_id) REFERENCES Comment(comment_id) ON DELETE CASCADE
);

-- ------------------------------------------------------------
-- 12. PLAYER_RATING  (fan ratings per match)
-- ------------------------------------------------------------
CREATE TABLE PlayerRating (
    rating_id    INT           UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    player_id    INT           UNSIGNED NOT NULL,
    match_id     INT           UNSIGNED NOT NULL,
    user_id      INT           UNSIGNED NOT NULL,
    rating       TINYINT       UNSIGNED NOT NULL,
    created_at   TIMESTAMP     DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY uq_player_rating (player_id, match_id, user_id),
    CONSTRAINT chk_player_rating CHECK (rating BETWEEN 1 AND 5),
    CONSTRAINT fk_pr_player FOREIGN KEY (player_id) REFERENCES Player(player_id) ON DELETE CASCADE,
```

```sql
    CONSTRAINT fk_pr_match  FOREIGN KEY (match_id)  REFERENCES `Match`(match_id)  ON DELETE CASCADE,
    CONSTRAINT fk_pr_user   FOREIGN KEY (user_id)   REFERENCES User(user_id)    ON DELETE CASCADE
);

-- ------------------------------------------------------------
-- 13. HIGHLIGHT
-- ------------------------------------------------------------
CREATE TABLE Highlight (
    highlight_id  INT          UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    match_id      INT          UNSIGNED NOT NULL,
    title         VARCHAR(255) NOT NULL,
    video_url     VARCHAR(500) NOT NULL,
    added_by      INT          UNSIGNED,    -- Admin user_id
    created_at    TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_hl_match FOREIGN KEY (match_id) REFERENCES `Match`(match_id) ON DELETE CASCADE,
    CONSTRAINT fk_hl_admin FOREIGN KEY (added_by) REFERENCES User(user_id) ON DELETE SET NULL
);

-- ------------------------------------------------------------
-- 14. FAN_EVENT
-- ------------------------------------------------------------
CREATE TABLE FanEvent (
    event_id    INT          UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    title       VARCHAR(255) NOT NULL,
    description TEXT,
    event_date  DATETIME     NOT NULL,
    location    VARCHAR(300),
    capacity    SMALLINT     UNSIGNED NOT NULL DEFAULT 100,
    created_by  INT          UNSIGNED,
    created_at  TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_fe_admin FOREIGN KEY (created_by) REFERENCES User(user_id) ON DELETE SET NULL
);

-- ------------------------------------------------------------
-- 15. EVENT_REGISTRATION  (junction: User <-> FanEvent)
-- ------------------------------------------------------------
CREATE TABLE EventRegistration (
    registration_id INT       UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    event_id        INT       UNSIGNED NOT NULL,
    user_id         INT       UNSIGNED NOT NULL,
    registered_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY uq_event_user (event_id, user_id),
```

```sql
    CONSTRAINT fk_er_event FOREIGN KEY (event_id) REFERENCES FanEvent(event_id) ON DELETE CASCADE,
    CONSTRAINT fk_er_user  FOREIGN KEY (user_id)  REFERENCES User(user_id)    ON DELETE CASCADE
);

-- ------------------------------------------------------------
-- 16. USER_FOLLOW_TEAM  (junction: User <-> Team)
-- ------------------------------------------------------------
CREATE TABLE UserFollowTeam (
    user_id     INT UNSIGNED NOT NULL,
    team_id     INT UNSIGNED NOT NULL,
    followed_at TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (user_id, team_id),
    CONSTRAINT fk_uft_user FOREIGN KEY (user_id) REFERENCES User(user_id) ON DELETE CASCADE,
    CONSTRAINT fk_uft_team FOREIGN KEY (team_id) REFERENCES Team(team_id) ON DELETE CASCADE
);

-- ------------------------------------------------------------
-- 17. USER_FOLLOW_PLAYER  (junction: User <-> Player)
-- ------------------------------------------------------------
CREATE TABLE UserFollowPlayer (
    user_id     INT UNSIGNED NOT NULL,
    player_id   INT UNSIGNED NOT NULL,
    followed_at TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (user_id, player_id),
    CONSTRAINT fk_ufp_user   FOREIGN KEY (user_id)   REFERENCES User(user_id)     ON DELETE CASCADE,
    CONSTRAINT fk_ufp_player FOREIGN KEY (player_id) REFERENCES Player(player_id) ON DELETE CASCADE
);

-- ------------------------------------------------------------
-- 18. SYNC_LOG
-- ------------------------------------------------------------
CREATE TABLE SyncLog (
    log_id          INT          UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    sync_type       VARCHAR(100) NOT NULL,   -- e.g. "live_scores", "standings"
    started_at      TIMESTAMP    NOT NULL,
    finished_at     TIMESTAMP    NULL,
    records_fetched INT          UNSIGNED DEFAULT 0,
    records_upserted INT         UNSIGNED DEFAULT 0,
    status          ENUM('running','success','failed') DEFAULT 'running',
    error_message   TEXT
```

```sql
);

-- ==============================================================
--  INDEXES
-- ==============================================================
CREATE INDEX idx_match_status      ON `Match`(status);
CREATE INDEX idx_match_datetime    ON `Match`(match_datetime);
CREATE INDEX idx_player_team       ON Player(team_id);
CREATE INDEX idx_comment_entity    ON Comment(entity_type,
   entity_id);
CREATE INDEX idx_review_match      ON Review(match_id);
CREATE INDEX idx_highlight_match   ON Highlight(match_id);
```

Listing 1: Sportify – MySQL Schema (DDL)

6.4 Key Design Decisions

- **JSON columns** (`extra_data`, `extra_stats`) store sport-specific raw payload without breaking normalisation of well-known columns.
- **ENUM status** on `Match` ensures only valid transitions are stored.
- **Composite UNIQUE keys** on junction tables prevent duplicate follows and reviews at the database level, not just application level.
- `external_api_id` columns on Match, Team, Player, and League enable idempotent upserts.
- `Comment.parent_id` self-referencing FK enables threaded comments (one level deep enforced at application layer).

7 API Design

All endpoints are prefixed with `/api/v1`. Authentication header: `Authorization: Bearer <access_token>`.
Access levels: **P** = Public, **A** = Authenticated, **M** = Team Manager+, **Ad** = Admin only.

Method	Endpoint	Auth	Description
Authentication			
POST	`/auth/register`	P	Register new user
POST	`/auth/login`	P	Login, returns tokens
POST	`/auth/refresh`	A	Refresh access token
POST	`/auth/logout`	A	Revoke refresh token
POST	`/auth/password-reset`	P	Request password reset email
Matches			
GET	`/matches`	P	List matches (filterable)
GET	`/matches/live`	P	Live matches only
GET	`/matches/{id}`	P	Match detail + stats
GET	`/matches/{id}/highlights`	P	Highlights for a match
Teams			
GET	`/teams`	P	List teams
GET	`/teams/{id}`	P	Team profile
GET	`/teams/{id}/players`	P	Squad list
PUT	`/teams/{id}/squad`	M	Update squad details
Players			
GET	`/players/{id}`	P	Player profile + stats
PUT	`/players/{id}/injury`	M	Update injury status
POST	`/players/{id}/rate`	A	Submit player rating
Leagues & Standings			
GET	`/leagues`	P	List leagues
GET	`/leagues/{id}/standings`	P	League table
Reviews & Comments			
GET	`/matches/{id}/reviews`	P	All reviews for a match
POST	`/matches/{id}/reviews`	A	Submit review
POST	`/comments`	A	Post a comment
PUT	`/comments/{id}`	A	Edit own comment
DELETE	`/comments/{id}`	A	Delete own comment

Method	Endpoint	Auth	Description
DELETE	`/admin/comments/{id}`	Ad	Admin hide/delete
Follow System			
POST	`/follow/team/{id}`	A	Follow a team
DELETE	`/follow/team/{id}`	A	Unfollow a team
POST	`/follow/player/{id}`	A	Follow a player
DELETE	`/follow/player/{id}`	A	Unfollow a player
GET	`/users/me/feed`	A	Personalised dashboard feed
Fan Events			
GET	`/events`	P	List fan events
POST	`/events`	Ad	Create fan event
POST	`/events/{id}/register`	A	Register for event
Admin			
GET	`/admin/users`	Ad	List all users
PUT	`/admin/users/{id}/role`	Ad	Change user role
DELETE	`/admin/users/{id}`	Ad	Suspend / delete user
GET	`/admin/sync-logs`	Ad	View sync job history

8 Authentication & Security Design

8.1 Authentication Flow

1. User submits credentials to `POST /auth/login`.
2. Flask verifies email existence, then compares submitted password to stored bcrypt hash.
3. On success, Flask generates:
   - **Access Token** — JWT signed HS256, payload: `{user_id, role, exp: now+15min}`.
   - **Refresh Token** — random 64-byte token, hashed (SHA-256) and stored in `RefreshToken` table.
4. Access Token sent in response body; Refresh Token set as `httpOnly; Secure; SameSite=Strict` cookie.
5. Frontend stores Access Token in memory (not `localStorage`) to mitigate XSS.
6. On 401, frontend calls `POST /auth/refresh` to get a new Access Token using the cookie.

8.2 RBAC Enforcement

- A Flask `@require_role(*roles)` decorator validates the JWT and checks `role` claim before executing any protected route handler.
- Team Managers can only modify data belonging to their `team_managed` (checked server-side).

8.3 Input Validation & Injection Prevention

- All request bodies validated via **Marshmallow** schemas; invalid requests return 422.
- All database access uses **SQLAlchemy parameterised queries** — no string-concatenated SQL.
- HTML-escaping applied to all user-generated text before storage to prevent stored XSS.

8.4 Rate Limiting

Flask-Limiter applies the following limits per IP:

- Public endpoints: 60 req / min.
- Auth endpoints (`/login`, `/register`): 10 req / min.
- Write endpoints: 30 req / min.

9 External API Integration

9.1 Candidate APIs

API	Sports Covered	Relevant Endpoints
API-Sports	Football, Cricket	`/fixtures`, `/standings`, `/players`
Cricbuzz (RapidAPI)	Cricket	`/matches/live`, `/series`
TheSportsDB	Multi-sport	`/lookupteam`, `/eventslast`

9.2 Data Sync Strategy

Data Type	Frequency	Strategy
Live match scores	Every 60 seconds	Poll only during live window (match_status = live)
Fixtures / schedule	Every 6 hours	Full refresh of upcoming 7 days
Standings	Every 6 hours	Replace season standings
Player profiles	Once / day	Incremental update by external_api_id
Post-match statistics	30 min after FT	Fetch once and lock

9.3 Upsert Pattern

```sql
INSERT INTO `Match`
    (league_id, home_team_id, away_team_id, match_datetime, status,
     home_score, away_score, elapsed_time, external_api_id, updated_at)
VALUES
    (:league_id, :home_id, :away_id, :dt, :status,
     :h_score, :a_score, :elapsed, :ext_id, NOW())
ON DUPLICATE KEY UPDATE
    status        = VALUES(status),
    home_score    = VALUES(home_score),
    away_score    = VALUES(away_score),
    elapsed_time  = VALUES(elapsed_time),
    updated_at    = NOW();
```

Listing 2: Idempotent Upsert Example (Match)

10 Constraints & Assumptions

10.1 Constraints

- The system targets an academic environment; production SLA guarantees are not required beyond the grading period.
- External API free-tier rate limits (typically 100 req/day) must not be exceeded; the scheduler must track request counts.
- The database must run on MySQL 8.x; no NoSQL alternatives are permitted for primary data storage (DBMS course constraint).
- All schema changes must go through Alembic migration files tracked in the Git repository.

10.2 Assumptions

- A stable internet connection is available for the server to call external APIs.
- The external APIs return consistent JSON schemas; if schemas change, a mapping layer update is required.
- Only one Team Manager is assigned per team at any time.
- Highlight videos are hosted externally (YouTube); Sportify stores only the URL.
- Email delivery for password reset relies on a configured SMTP service (e.g. Gmail App Password or SendGrid).

11 Use Cases (Summary)

UC ID	Use Case	Actor	Brief Description
UC-01	Register Account	Visitor	Visitor submits email + password; account created with Fan role.
UC-02	Login	All Users	User provides credentials; system returns JWT.
UC-03	View Live Scores	Visitor/Fan	User opens live scores page; data auto-refreshes every 30 s.
UC-04	Follow a Team	Fan	Fan clicks Follow; junction row inserted in UserFollowTeam.
UC-05	Post Match Review	Fan	Fan submits star rating + text for a finished match.
UC-06	Update Injury Status	Team Manager	Manager marks player as injured; timestamp recorded.
UC-07	Manage Users	Admin	Admin views user list, suspends account, or changes role.
UC-08	Sync Live Data	System (Scheduler)	Scheduler polls API, transforms, upserts into MySQL, logs run.
UC-09	Register for Event	Fan	Fan registers; capacity decremented; duplicate prevented by UNIQUE key.
UC-10	Add Highlight Video	Admin	Admin submits match ID + YouTube URL; stored in Highlight table.

12 Testing Strategy

Test Type	Tool	Scope
Unit Tests	pytest	Individual Flask route handlers, DB helper functions
Integration Tests	pytest + test DB	API → MySQL round-trips
Authentication Tests	pytest	JWT issuance, token refresh, RBAC enforcement
DB Constraint Tests	MySQL client + pytest	Duplicate key, FK violation, CHECK constraint behaviour
API Contract Tests	Postman / pytest	Every endpoint against OpenAPI schema
Scheduler Tests	pytest + mock	External API mocked; upsert logic verified

13 Project Timeline (Indicative)

Week	Milestone	Deliverable
1–2	Requirements & Design	SRS document, ER Diagram, Wireframes
3–4	Database Implementation	DDL scripts, seed data, Alembic migrations
5–6	Backend API Development	Flask endpoints (auth, matches, teams, players)
7	External API Integration	Scheduler, upsert logic, SyncLog
8	Frontend Development	ReactJS SPA: live scores, standings, player pages
9	Community Features	Reviews, comments, follow system, events
10	Testing & QA	Unit, integration, and regression tests
11	Polish & Documentation	OpenAPI docs, README, deployment notes
12	Demo & Submission	Final presentation and code submission